

Handwriting Autocomplete System

Deep Learning Project Report

Abhijeet, Akshat, Devyansh A. & Raghav

Contents

1. Executive Summary	1
2. Project Overview	1
3. Phase 1: Optical Character Recognition (OCR)	2
4. Phase 2: Next Word Prediction	4
5. Phase 3: Handwriting Style Transfer	4
6. Integrated Pipeline	6

1. Executive Summary

This project aims to develop a **Handwriting Autocomplete System** that not only predicts the next word in a sentence but also generates that prediction in the user's unique handwriting style. The system integrates three major deep learning components: **Optical Character Recognition (OCR)** to understand the user's input, **Next Word Prediction (NWP)** to anticipate the user's intent, and **Handwriting Style Transfer** to render the prediction seamlessly.

By leveraging state-of-the-art architectures like CNN-LSTMs, Transformers (GPT-2), and Generative Adversarial Networks (HiGAN+), we address the complex challenge of maintaining semantic coherence while preserving the visual authenticity of handwritten text.

2. Project Overview

Problem Statement

Handwriting remains a personal and expressive form of communication. However, digital tools often strip away this individuality, converting handwritten notes into standardized fonts. Our goal is to bridge this gap by creating an intelligent system that: 1. **Reads** handwritten text despite variations in style, noise, and distortion. 2. **Predicts** the next logical word(s) to assist the writer. 3. **Imitates** the writer's specific style (slant, pressure, stroke width) to generate the predicted word.

Core Components

The project is divided into three distinct phases: * **Phase 1: OCR** - Converting handwritten images to text. * **Phase 2: Next Word Prediction** - Generating semantic completions. * **Phase 3: Style Transfer** - Rendering text in a target handwriting style.

3. Phase 1: Optical Character Recognition (OCR)

3.1 Overview

The first step in our pipeline is to accurately digitize the user's handwritten input. We implemented a robust Handwritten Text Recognition (HTR) system capable of segmenting full sentences into words and recognizing them individually.

3.2 Word Segmentation

Before recognition, the input sentence image must be segmented into individual words. We developed a **7-step computer vision pipeline**.

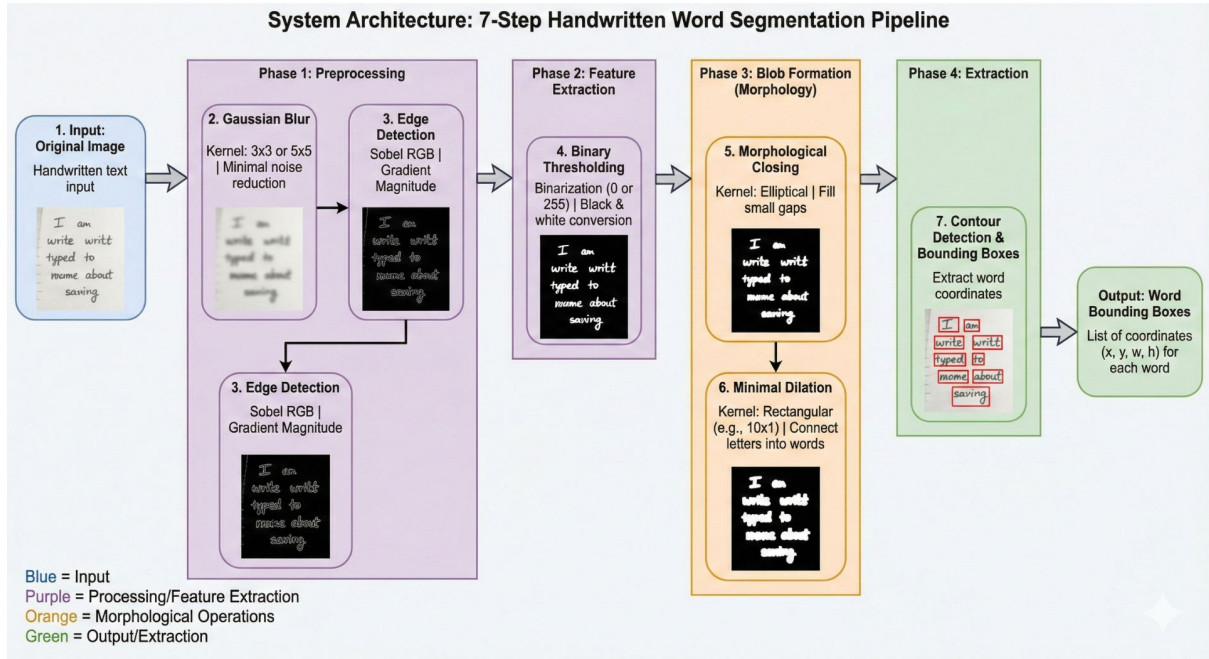


Figure 1: The 7-step word segmentation pipeline.

The pipeline handles various challenges such as uneven spacing and noise, ensuring that the recognizer receives clean, isolated word images.

3.3 Model Architecture: CNN-LSTM

For the recognition task, we employed a **CRNN (Convolutional Recurrent Neural Network)** architecture.

- **CNN Backbone:** Extracts spatial features (strokes, curves, shapes) from the word images.
- **BiLSTM (Bidirectional LSTM):** Captures sequential dependencies, understanding the order of characters.
- **CTC (Connectionist Temporal Classification) Loss:** Handles the alignment between the input image sequence and the output text sequence, allowing for variable-length inputs.

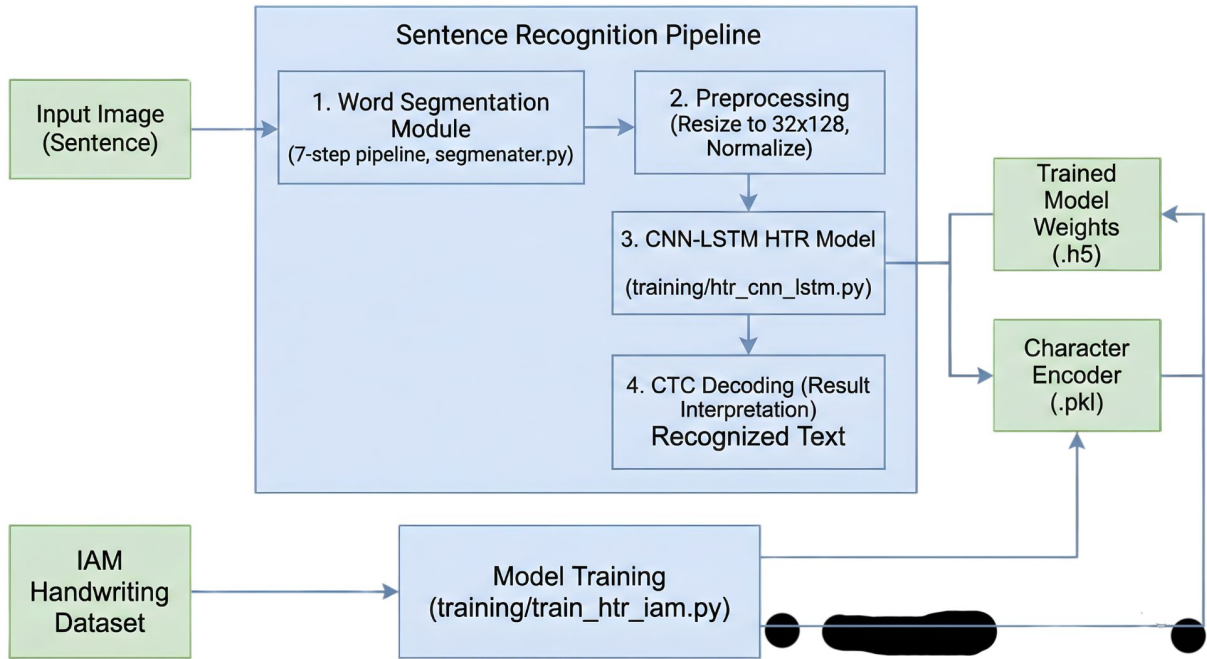


Figure 2: The OCR model architecture and processing flow.

3.4 Performance & Limitations

- **Metrics:** We evaluated the model using Character Error Rate (CER) and Word Error Rate (WER).
- **Strengths:** Performs well on standard datasets like IAM.
- **Limitations:** Struggles with highly cursive text, extreme noise, or rare words not present in the training vocabulary.

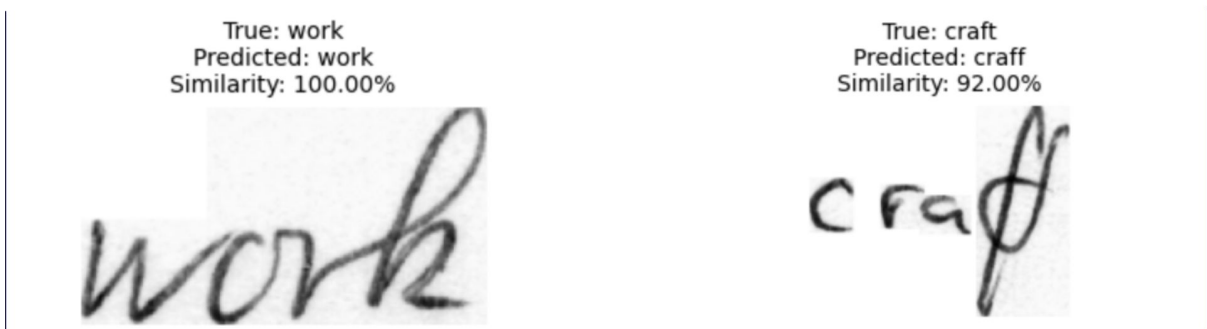


Figure 3: Example of OCR output from handwritten input.

4. Phase 2: Next Word Prediction

4.1 Overview

Once the text is recognized, the system predicts the most likely next word. We utilized the **GPT-2 (Generative Pre-trained Transformer)** architecture for this task, leveraging its powerful language modeling capabilities.

4.2 Methodology

We followed a “from-scratch” implementation approach inspired by Andrej Karpathy, while also utilizing pre-trained checkpoints due to hardware constraints.

- **Model Size:** 124 Million parameters.
- **Context Length:** 1024 tokens.
- **Vocabulary:** 50,257 BPE tokens.

4.3 Key Optimizations & Implementation Details

To ensure efficiency and stability, we implemented several advanced techniques:

1. **Weight Tying:** The input embedding layer and the output head share weights. This reduces the model size by ~30% (saving ~38 million parameters) and enforces semantic logic where the input vector for “cat” matches the output prediction vector for “cat”.
 2. **Residual Stream Initialization ($1/\sqrt{2N}$):** To prevent variance explosion in deep networks, we scaled the weights of residual layers. This ensures a “clean” signal propagation and stable training.
 3. **“Ugly Numbers” Optimization:** We padded the vocabulary size from 50,257 to **50,304**. Since 50,304 is divisible by 128, it allows CUDA kernels to fully saturate the GPU hardware, resulting in a ~4% speedup in training.
-

5. Phase 3: Handwriting Style Transfer

5.1 Overview

The final and most visually complex phase is generating the predicted word in the user’s handwriting style. We adopted a **Modified HiGAN+ (Handwriting Imitation GAN)** architecture, which excels at disentangling style from content.

5.2 Architecture: Modified HiGAN+

Our implementation introduces several key innovations over the original HiGAN+ paper to enhance realism and stability.

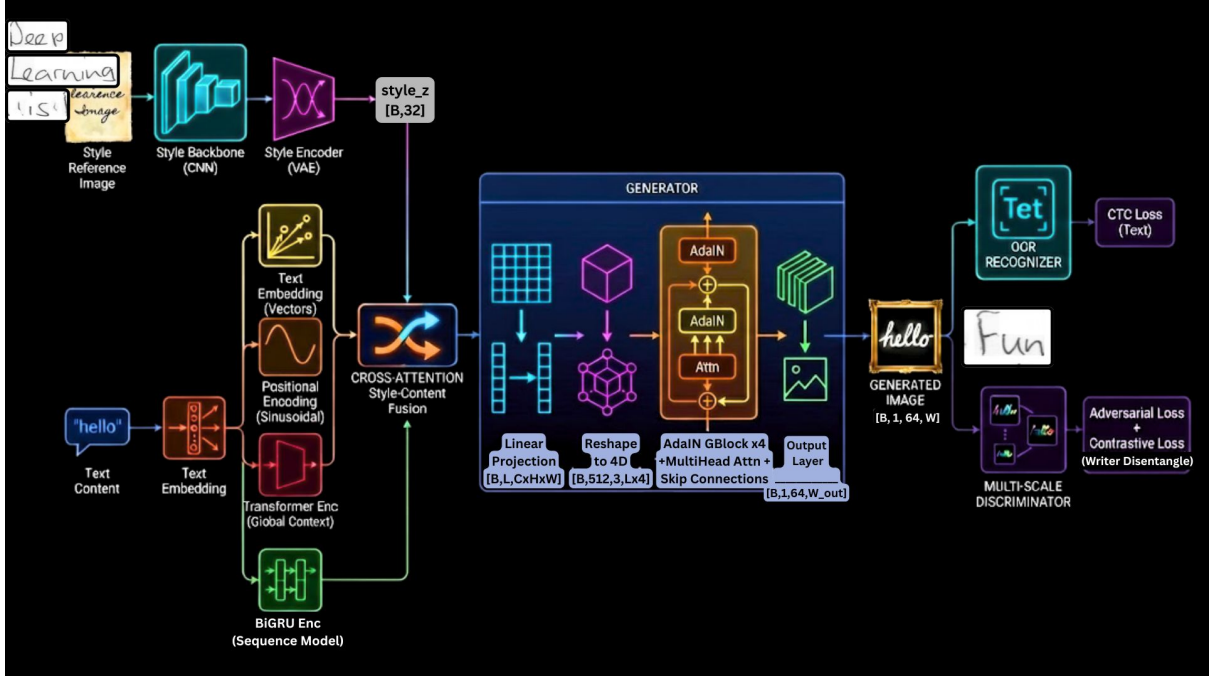


Figure 4: The Style Transfer pipeline architecture.

Key Innovations:

1. **Dual-Scale Discriminator:**
 - **Global Discriminator:** Evaluates overall word structure and consistency.
 - **Patch Discriminator:** Focuses on fine-grained details like ink texture and pen pressure.
2. **VAE Integration:** We integrated a Variational Autoencoder (VAE) into the style encoder. This allows for a continuous style manifold, enabling smooth interpolation between styles and reducing mode collapse.
3. **Gradient Penalty Balancing:** We implemented an adaptive mechanism to balance gradients from different losses (CTC, Writer ID, Reconstruction), ensuring no single objective dominates the training.
4. **Contextual Loss:** A loss function that measures feature distribution similarity without requiring strict pixel-level alignment, helping capture writer-specific stroke thickness and slant.

5.3 Performance

The model was evaluated using metrics such as **FID (Fréchet Inception Distance)** for image quality and **WIER (Writer Identification Error Rate)** for style consistency. The dual-scale discriminator significantly reduced blurriness, resulting in sharper character boundaries.

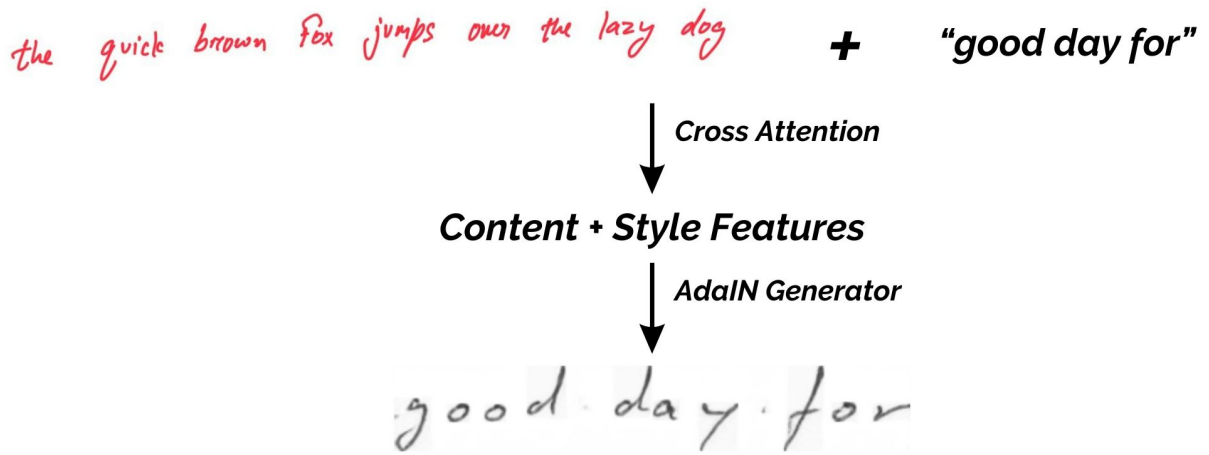


Figure 5: Generated samples showing the model’s ability to mimic different handwriting styles.

6. Integrated Pipeline

The full system operates as a seamless pipeline:

1. **Input:** User writes a sentence (e.g., “the Quick Brown Fox”).
2. **OCR:** The system reads the text.
3. **Prediction:** GPT-2 predicts the next word (e.g., “jumps”).
4. **Style Extraction:** The system analyzes the style of the input sentence.
5. **Generation:** HiGAN+ generates the word “jumps” using the extracted style.
6. **Output:** The user sees “jumps” appear in their own handwriting.

This end-to-end flow demonstrates the power of combining discriminative models (OCR), generative language models (GPT-2), and conditional image generation models (GANs) into a cohesive application.

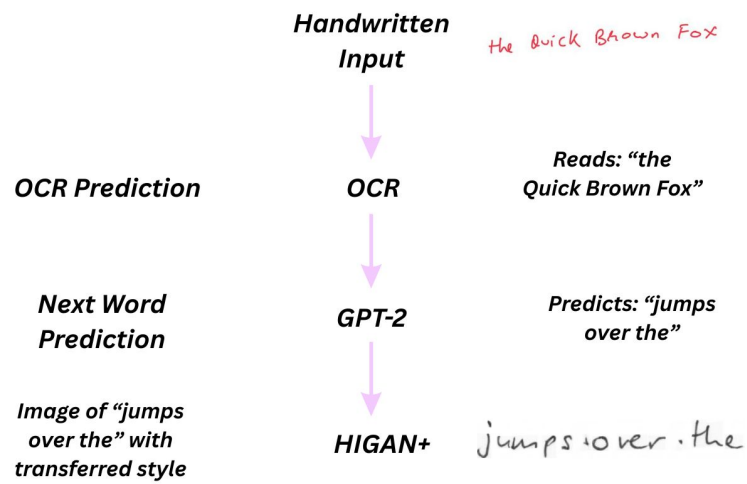


Figure 6: Example of the whole pipelin in action.